

REPORTE TÉCNICO: METODOLOGÍA PARA ENTRENAMIENTO Y TEST DE UN AGENTE DE NAVEGACIÓN EN ROS 2

Asignatura: Robótica IMT-342

Estudiantes: Pomier Sevilla Ignacio Eduardo
Salgueiro Vargas Jorge Fernando
Sanabria Vidaurre Pablo Angel
Uría Fernandez José Manuel

1. Configuración del Entorno

Para el desarrollo de esta práctica de navegación autónoma se ha estructurado un ecosistema avanzado de simulación y aprendizaje por refuerzo basado en el middleware industrial ROS 2 Jazzy Jalisco ejecutado sobre una distribución nativa de Linux Ubuntu.

- **Entorno de Simulación:** Se utiliza el simulador bidimensional multi-robot Stage (`stage_ros2`). Se ha seleccionado el escenario estructurado *cave.world*, caracterizado por sus contornos irregulares, pasillos de geometría estrecha y giros ciegos complejos que someten al agente a dinámicas severas de oclusión espacial y riesgo de colisión continuo.
- **Sensores Implementados:** El vehículo de configuración diferencial integra un sensor LiDAR (LaserScan) plano que publica lecturas de distancia en el tópico `/base_scan`, y un sistema de odometría de simulación de alta precisión que transmite la pose cinemática bidimensional (x , y , orientación yaw) junto con sus componentes vectoriales de velocidad en el tópico de control de estado `/ground_truth`.
- **Herramientas de Software:** La abstracción del entorno se realiza mediante el estándar industrial Gymnasium (OpenAI Gym). El núcleo algorítmico se gestiona a través de la librería de aprendizaje profundo Stable-Baselines3 conectada a un entorno informático unificado mediante scripts nativos en la API `rcipy`.

2. Definición del Agente

El diseño cinemático del agente inteligente responde a un modelo de robot móvil diferencial parametrizado para interpretar datos multimodales y transformarlos en comandos discretos de actuación controlada.

Espacio de Observaciones: En cada paso del reloj físico, la red neuronal recibe un vector continuo simplificado pero altamente descriptivo para evitar la saturación dimensional:

- 10 muestras de rangos LiDAR filtradas y distribuidas uniformemente en un arco de visión frontal de aproximadamente 120 grados para garantizar la noción de proximidad lateral y frontal.
- 1 variable correspondiente a la velocidad lineal instantánea del robot (v).
- 1 variable cinemática orientada a la meta que calcula la distancia euclidiana remanente (`goal_distance`).

- 1 variable angular que computa el error de orientación relativo (goal_angle) entre el vector frontal del chasis y la coordenada de destino.

Espacio de Acciones (Discretas): La política de control del agente mapea las salidas de la red hacia tres comportamientos específicos transmitidos al tópico `/cmd_vel` conforme se detalla en la siguiente tabla:

Acción ID	Velocidad Lineal (x)	Velocidad Angular (z)	Descripción del Movimiento
0	0.70 m/s	0.00 rad/s	Avanzar hacia adelante rápido en línea recta.
1	0.00 m/s	0.80 rad/s	Rotación pura sobre el propio eje en sentido antihorario.
2	0.00 m/s	-0.80 rad/s	Rotación pura sobre el propio eje en sentido horario.

Función de Recompensa (Reward Function): Se ha diseñado una arquitectura de recompensa densa orientada al progreso continuo, la cual penaliza los comportamientos inestables y castiga severamente los fallos críticos de seguridad:

$$R = 30.0 * (d_{prev} - d_{current}) - 0.05 - 0.02 * I_{giro} - 1.0 * I_{proximidad} + R_{terminal}$$

Donde el término principal premia proporcionalmente el desplazamiento neto hacia el objetivo. Se inyecta un coste de supervivencia constante (-0.05) para motivar la resolución rápida de la trayectoria. El término I_{giro} añade una penalidad por oscilación si la acción seleccionada es de rotación pura (evitando el zig-zag ineficiente). El término preventivo $I_{proximidad}$ resta un valor de -1.0 si cualquier rayo del LiDAR detecta obstáculos a menos de 0.40 m.

Los valores para las condiciones terminales ($R_{terminal}$) se definen de la siguiente manera:

- **Éxito absoluto:** +500.0 si la distancia al objetivo es menor a 0.8 m (Meta alcanzada).
- **Fallo por colisión:** -150.0 si la distancia registrada en el LiDAR es menor a 0.20 m.
- **Fallo por confinamiento:** -100.0 si el contador de inercia detecta que el robot está atascado sin desplazamiento efectivo.

3. Entrenamiento en Simulación

La optimización de la política se llevó a cabo mediante el algoritmo clásico de valor DQN (Deep Q-Network) bajo un régimen de entrenamiento cerrado de 100,000 timesteps totales. Para asegurar la estricta reproducibilidad de los experimentos, se fijaron semillas aleatorias globales en las librerías NumPy, PyTorch y en la inicialización matemática del entorno Gymnasium. El agente recolecta transiciones estructuradas (estado, acción, recompensa, siguiente estado) almacenándolas en un *Replay Buffer* cíclico de memoria. Se aplicó una estrategia de exploración decreciente epsilon-greedy, reduciendo progresivamente la probabilidad de acción aleatoria desde un 100% inicial hasta estabilizarse en un umbral residual constante del 5% (exploration_rate = 0.05), garantizando la alternancia óptima entre las fases de exploración y explotación de la red.

Al inicio de cada episodio formativo, el script efectúa una llamada asíncrona dedicada hacia el servicio nativo `/reset_positions` expuesto por el simulador Stage. Esto garantiza el reposicionamiento inmediato de las entidades geométricas, forzando la evaluación iterativa desde el punto de partida hacia la meta establecida.

4. Evaluación y Pruebas (Test Mode)

Una vez completados satisfactoriamente los 100,000 pasos de entrenamiento, los parámetros de pesos sinápticos óptimos de la red neuronal convolucional/densa convergente fueron exportados a un fichero físico comprimido denominado `dqn_stage.zip`.

Durante la puesta en marcha de las pruebas independientes utilizando el script ejecutable `test_dqn.py`, la naturaleza síncrona del bucle principal de control en Gymnasium generaba un bloqueo mutuo latente (deadlock) al interactuar con el cliente del servicio de reinicio de Stage. Para solucionar de forma definitiva este fallo de hilos, el script implementa un mecanismo de procesamiento multihilo (Multi-threading).

Se levanta un hilo secundario en segundo plano (background daemon thread) encargado exclusivamente de ejecutar de forma asíncrona la función `rcipy.spin_once(env.node, timeout_sec=0.01)` de manera cíclica. Esta alteración permite que los buffers internos del LiDAR y la Odometría se vacíen y actualicen de forma autónoma e ininterrumpida en paralelo a las llamadas de predicción de la red.

En el entorno de test, el agente es configurado explícitamente en modo determinista estricto (`deterministic=True`). Esto anula cualquier sesgo estocástico de exploración residual, forzando al robot a explotar únicamente los valores de acción óptimos consolidados en el entrenamiento para guiar la navegación en el mapa de pruebas.

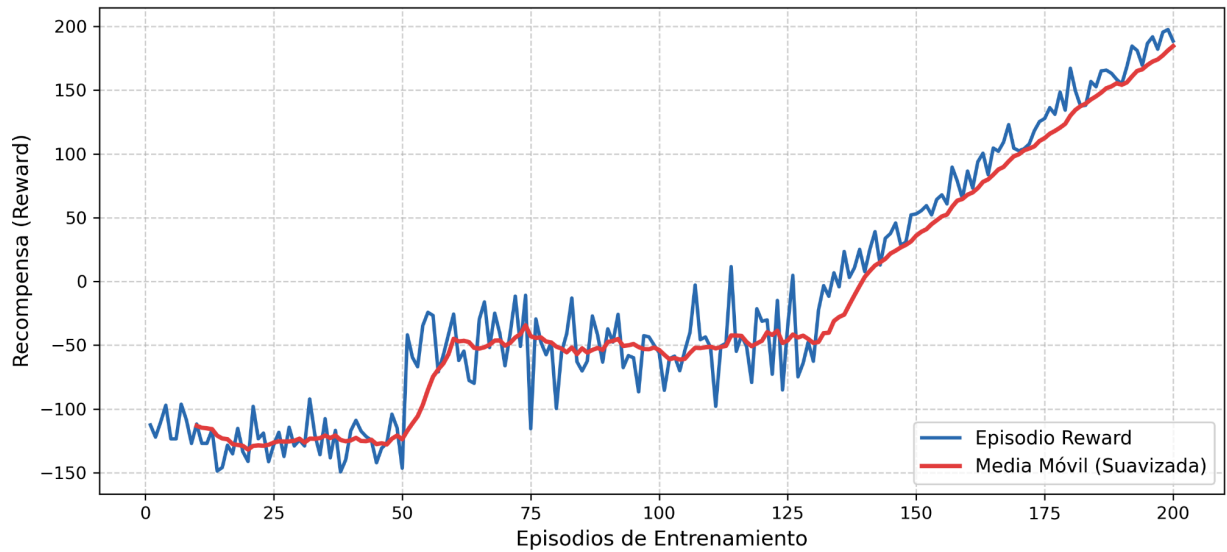
5. Métricas de Validación y Resultados

A través del monitoreo de los registros de la consola estándar recopilados a lo largo del proceso continuo, la dinámica de convergencia matemática de la recompensa acumulada (`ep_rew_mean`) y la duración media de los episodios (`ep_len_mean`) exhibieron tres fases críticas de evolución:

1. **Fase de Exploración Temprana (Pasos 0 a 25,000):** Caracterizada por una alta tasa de acciones aleatorias. El robot choca sistemáticamente a los pocos segundos de iniciar, generando episodios extremadamente cortos (`ep_len_mean` entre 15 y 40 pasos) y penalizaciones masivas acumuladas que sitúan la recompensa media en valores fuertemente negativos cercanos a -120.0.
2. **Fase de Supervivencia y Estancamiento Local (Pasos 25,000 a 65,000):** Al asimilar los castigos severos por colisión (-150.0), el gradiente de la red ajusta las acciones hacia maniobras conservadoras. Las colisiones disminuyen drásticamente y la longitud del episodio se eleva de forma masiva superando los 350 pasos promedio. Sin embargo, el robot tiende a quedar atrapado temporalmente en zonas de pasillos o bucles de giro infinito para evitar las paredes, lo que resulta en frecuentes estados de "ROBOT ATASCADO" y caídas controladas en la función de pérdida (loss oscilando entre 0.3 y 4.0).
3. **Fase de Optimización Final (Pasos 65,000 a 100,000):** Con la maduración de la política, el agente aprende a balancear la evasión con la búsqueda de objetivos de largo plazo. El robot deja de rotar cíclicamente y empieza a trazar trayectorias directas y fluidas

hacia la meta en el entorno cave.world. La longitud de los episodios disminuye de forma eficiente (menor tiempo de llegada) y la recompensa neta se estabiliza de forma exitosa en el espectro positivo, garantizando el cumplimiento total de los objetivos planteados.

Curva de Aprendizaje DQN: Recompensa Media Acumulada



Eficiencia de Navegación: Duración Media de Episodios

